

Text Classification Using Machine Learning

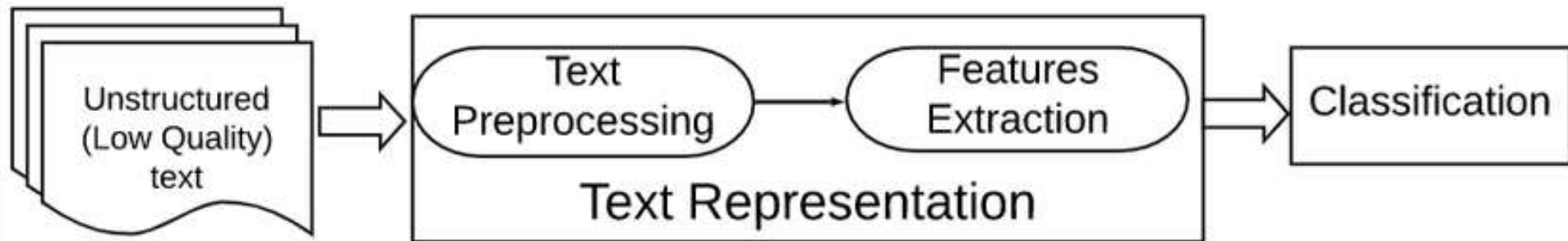
H.M. Samadhi Chathuranga Rathnayake



What is text classification?

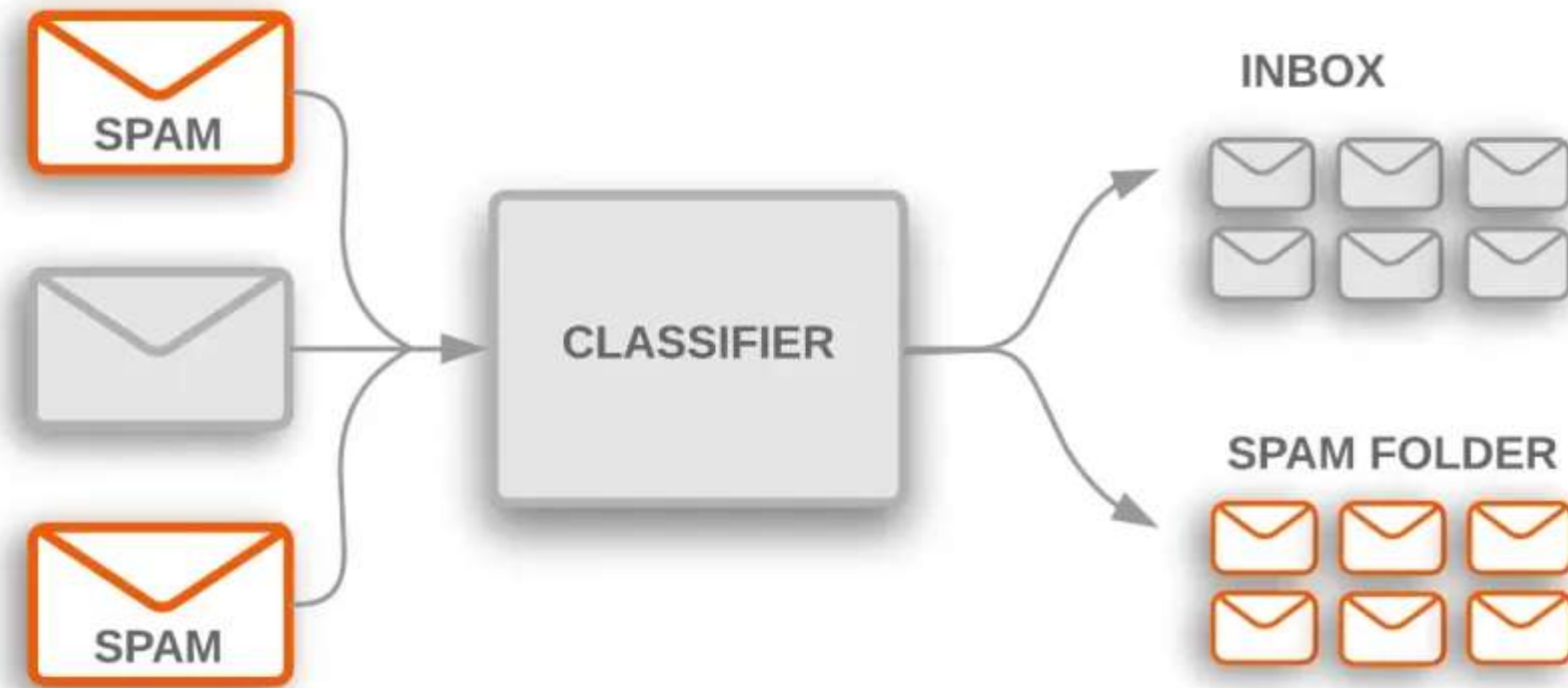
- Text classification is a common NLP task used to solve business problems in various fields.
- The goal of text classification is to categorize or predict a class of unseen text documents, often with the help of supervised machine learning.
- Similar to a classification algorithm that has been trained on a tabular dataset to predict a class, text classification also uses supervised machine learning.
- The fact that text is involved in text classification is the main distinction between the two.

Text classification pipeline



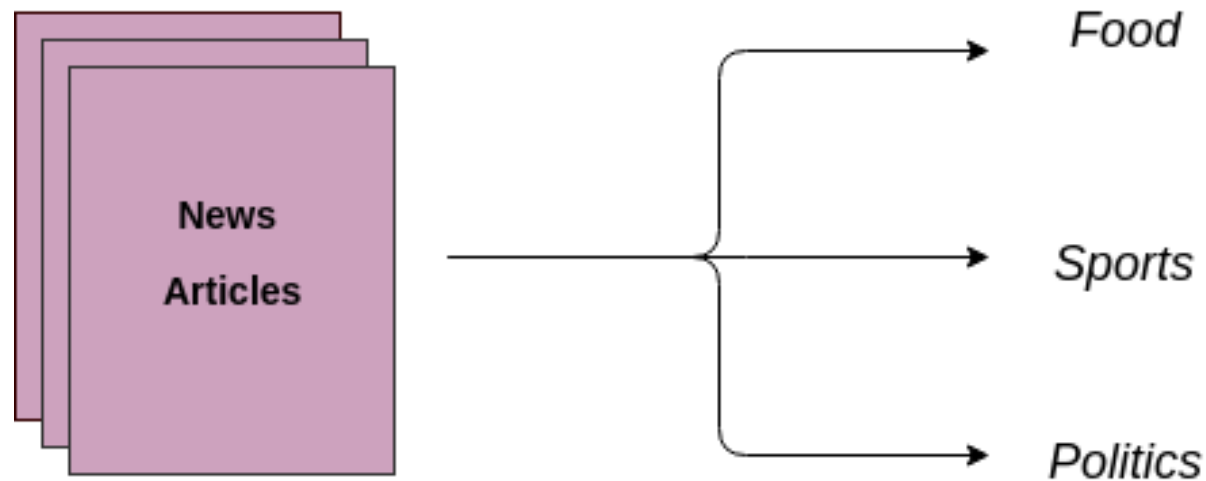
Use cases and applications

- Spam classification



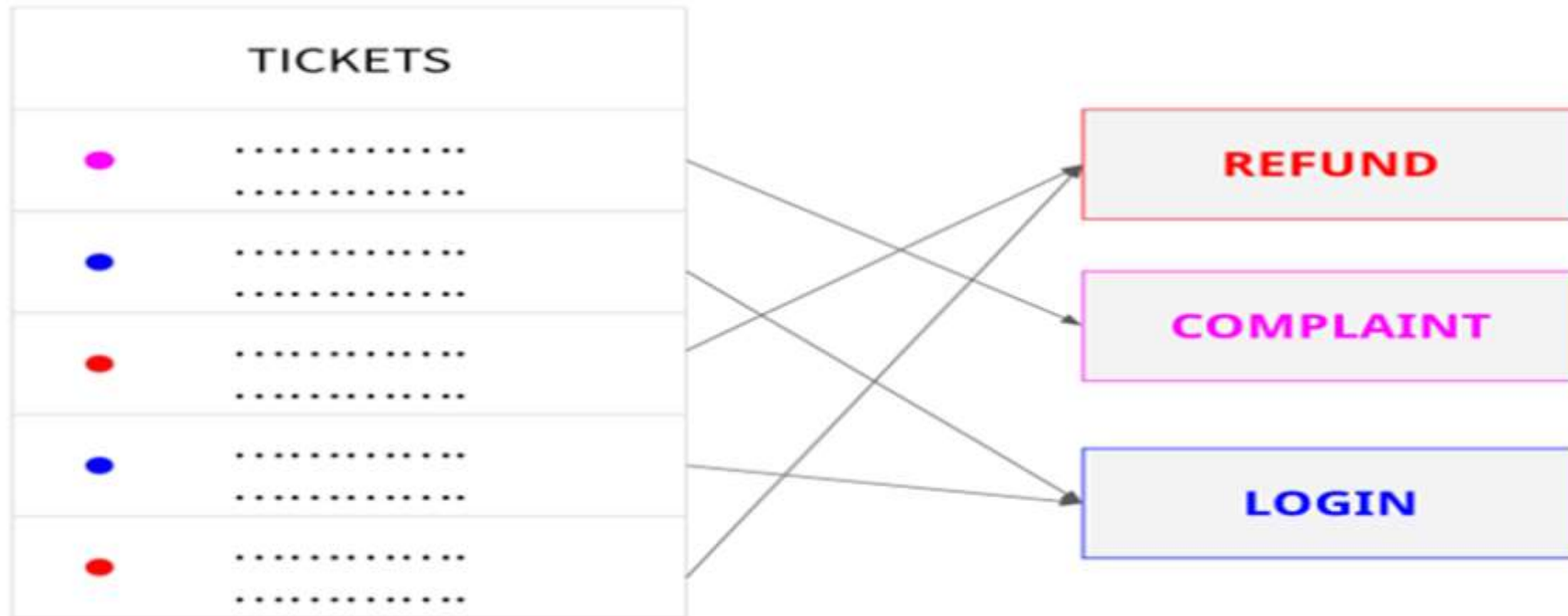
Use cases and applications

- Classifying news articles and blogs



Use cases and applications

- Categorize customer support requests



Types of text classification

- There are mainly two types of text classification systems
 - Rule-based text classification
 - Machine learning-based text classification

Rule-based text classification

- Rule-based techniques use a set of manually constructed language rules to categorize text into categories or groups.
- These rules tell the system to classify text into a particular category based on the content of a text by using semantically relevant textual elements.
- For example, imagine you have tons of new articles, and your goal is to assign them to relevant categories such as Sports, Politics, Economy, etc.
- With a rule-based classification system, you will do a human review of a couple of documents to come up with linguistic rules like this one:
- **If the document contains words such as money, dollar, GDP, or inflation, it belongs to the Politics group (class).**

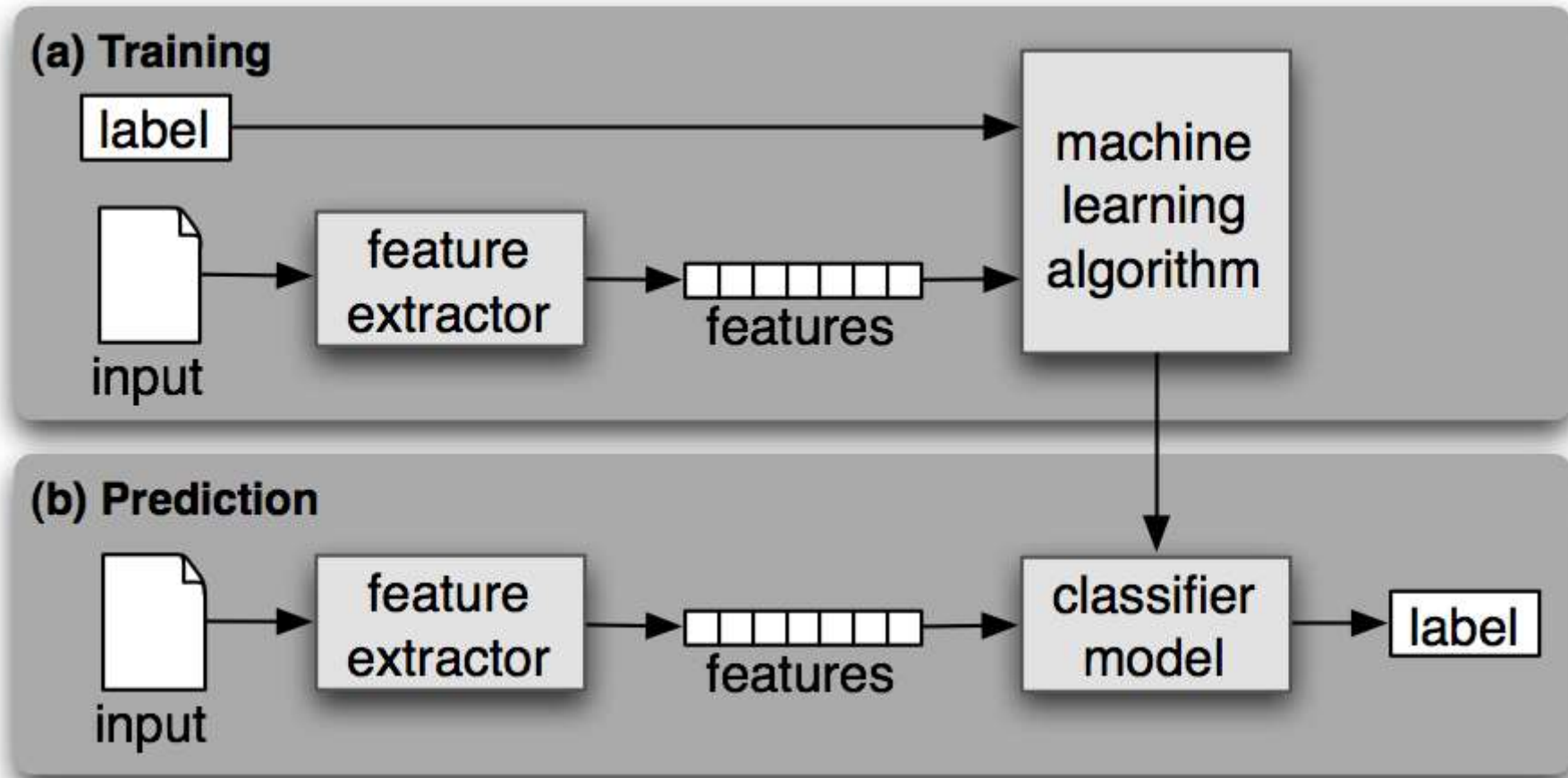
Rule-based text classification

- Rule-based systems can be refined over time and are understandable to humans.
- However, there are certain drawbacks to this strategy.
 - These systems, to begin with, demand in-depth expertise in the field.
 - They take a lot of time since creating rules for a complicated system can be difficult and frequently necessitates extensive study and testing.
 - Given that adding new rules can alter the outcomes of the pre-existing rules, rule-based systems are also challenging to maintain and do not scale effectively.

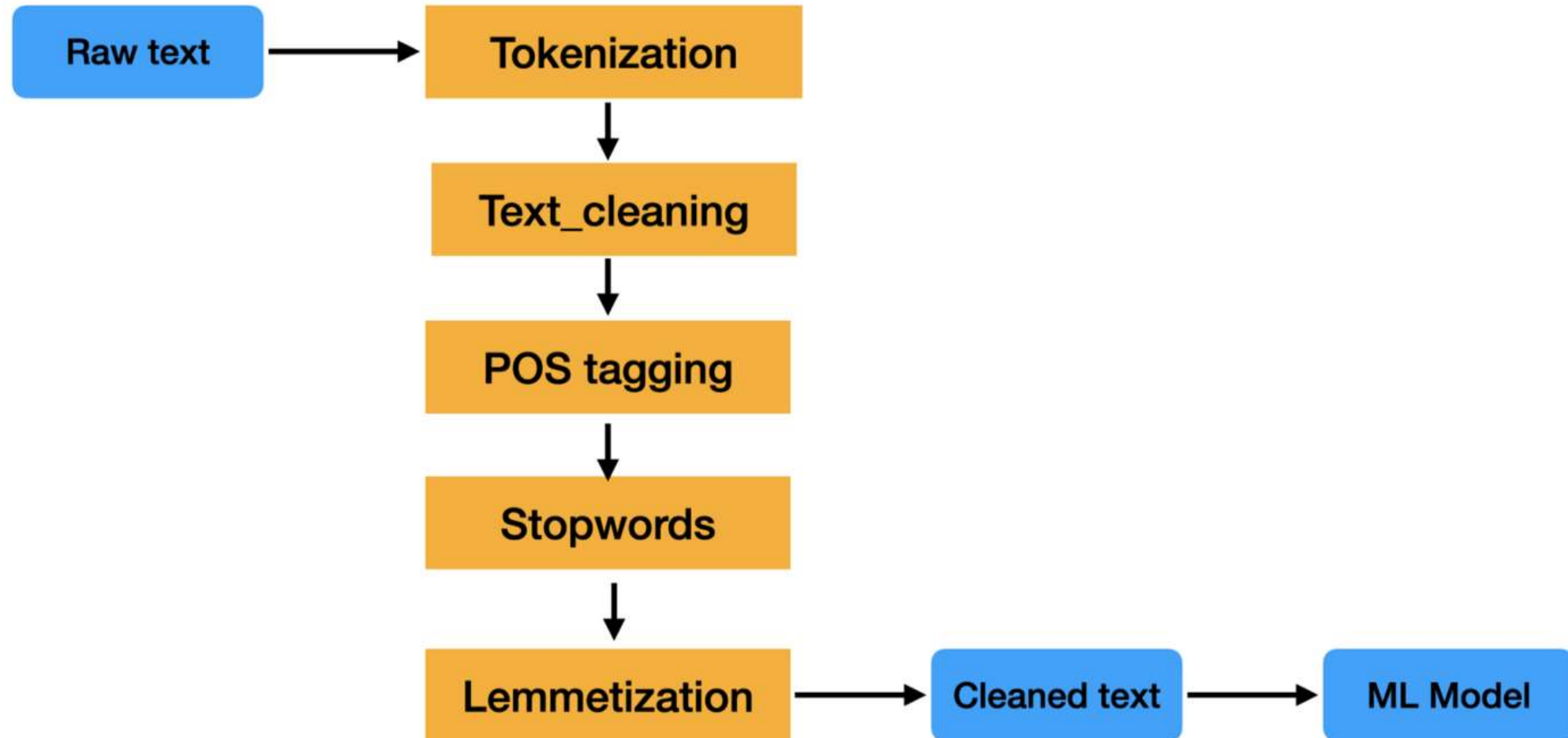
Machine learning-based text classification

- Machine learning-based text classification is a supervised machine learning problem.
- It learns the mapping of input data (raw text) with the labels (also known as target variables).
- This is similar to non-text classification problems where we train a supervised classification algorithm on a tabular dataset to predict a class, with the exception that in text classification, the input data is raw text instead of numeric features.

Machine learning-based text classification



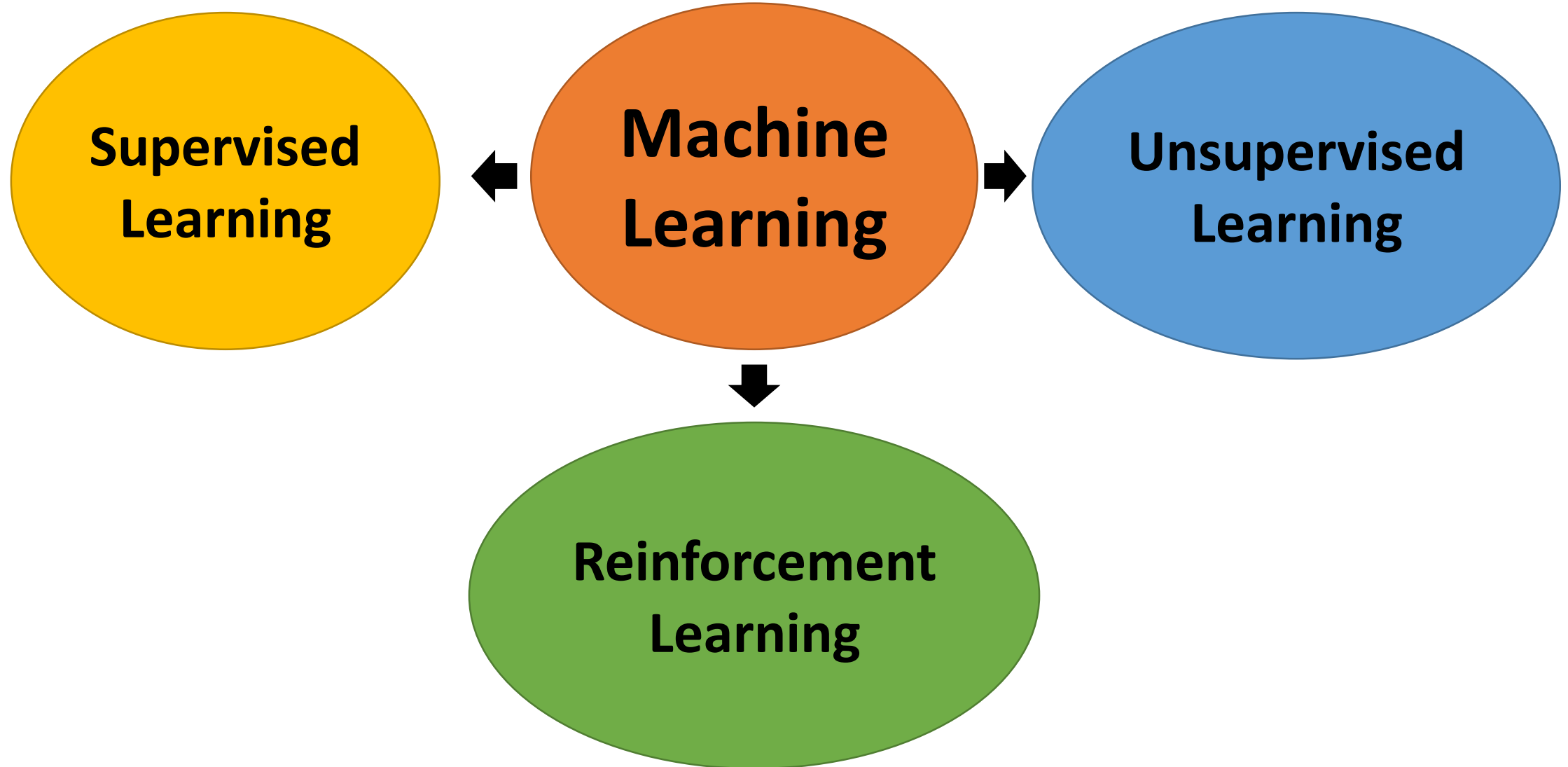
Text preprocessing



Machine Learning

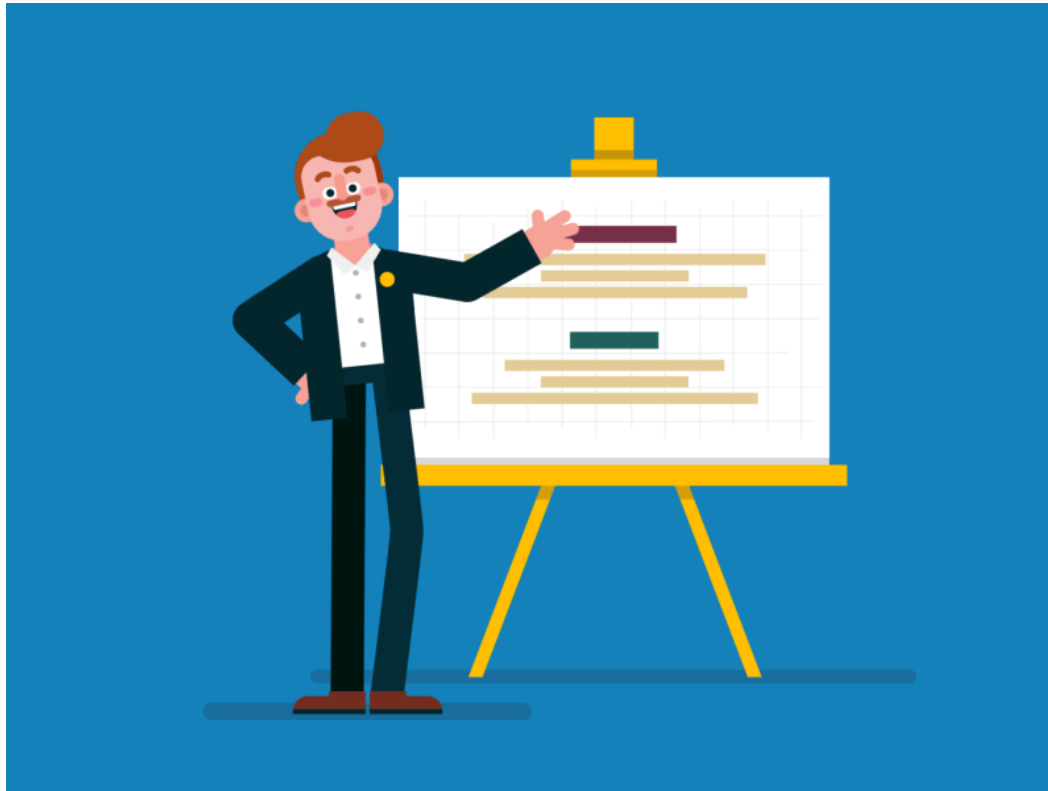
- Machine learning is like teaching computers to learn from experience. Instead of being explicitly programmed to perform a task, machines are trained using vast amounts of data to recognize patterns and make decisions on their own.
- Imagine showing a computer many pictures of cats and dogs until it learns to distinguish between them.
- Once trained, it can identify new cats and dogs it has never seen before.

Machine Learning



Supervised Learning

- In Supervised Learning, teacher shows how to do
- Students learn by understanding the inputs & outputs



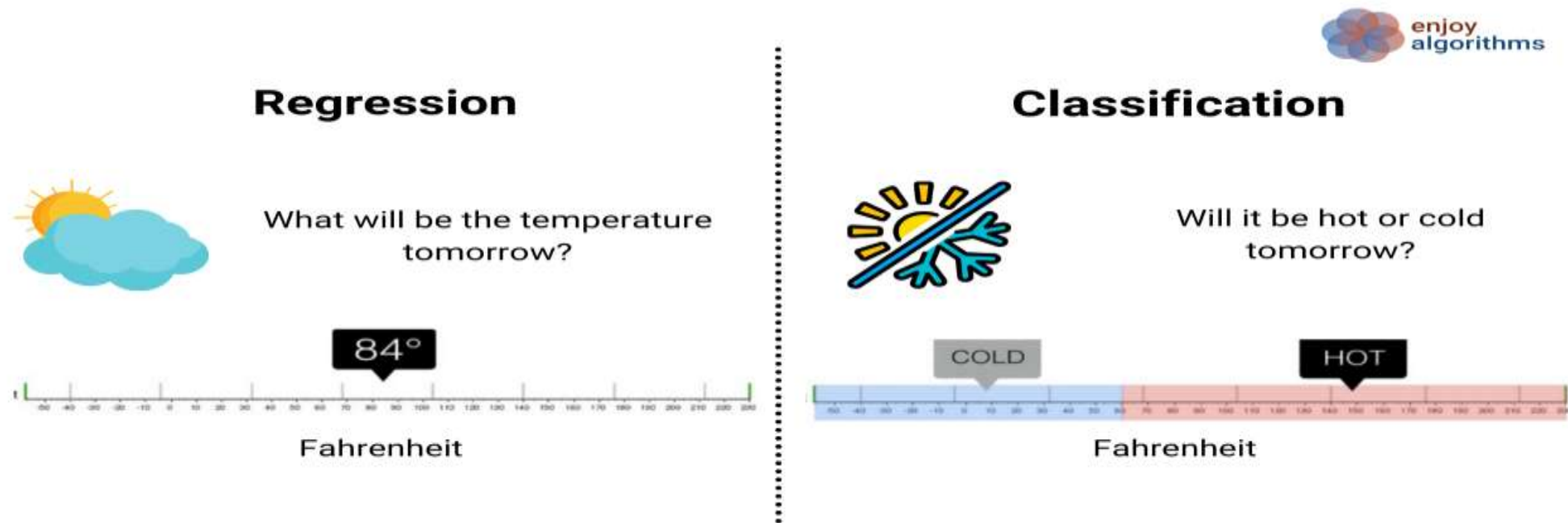
Supervised Learning

- Same thing is applied here.
- The model will be trained with inputs and related outputs.
- After training it can make decisions based on predictions on unseen situations.



Supervised Learning

- In Supervised Learning there are two areas
 - Regression
 - Classification



Classification Machine Learning Algorithms

Logistic
Regression

Naive
Bayes

k-Nearest
Neighbors

Categorical
Variable
Decision Tree

Continuous
Variable
Decision Tree

Random
Forest

Support
Vector Machine

Feature extraction

- The two most common methods for extracting feature from text or in other words converting text data (strings) into numeric features so machine learning model can be trained are:
 - Bag of Words
 - Tf-IDF.

Bag of Words

- A bag of words (BoW) model is a simple way of representing text data as numeric features.
- It involves creating a vocabulary of known words in the corpus and then creating a vector for each document that contains counts of how often each word appears.

Bag of Words

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

TF-IDF

- TF-IDF stands for term frequency-inverse document frequency, and it is another way of representing text as numeric features.
- There are some shortcomings of the Bag of Words (BoW) model that Tf-IDF overcomes.
- The TF-IDF model is different from the bag of words model in that it takes into account the frequency of the words in the document, as well as the inverse document frequency.
- This means that the TF-IDF model is more likely to identify the important words in a document than the bag of words model.

TF-IDF

	Terms in documents				
	Term 1	Term 2	...	Term k	Label (Y)
Doc 1	TF-IDF11	TF-IDF12	...	TF-IDF1k	Class1
Doc 2	TF-IDF21	TF-IDF22	...	TF-IDF2k	Class2
Doc 3	TF-IDF31	TF-IDF32	...	TF-IDF3k	Class2
⋮	⋮	⋮	...	⋮	⋮
Doc n	TF-IDFn1	TF-IDFn2	...	TF-IDFnk	Class1

TF-IDF

	Terms in documents				
	Term 1	Term 2	...	Term k	Label (Y)
Doc 1	TF-IDF11	TF-IDF12	...	TF-IDF1k	Class1
Doc 2	TF-IDF21	TF-IDF22	...	TF-IDF2k	Class2
Doc 3	TF-IDF31	TF-IDF32	...	TF-IDF3k	Class2
⋮	⋮	⋮	...	⋮	⋮
Doc n	TF-IDFn1	TF-IDFn2	...	TF-IDFnk	Class1



Model Evaluation

- The mainly discussed two techniques for evaluating a model in regression can be used for evaluating in the classification too.
 - Validation Set Approach
 - Cross Validation
- Using these approaches, we can measure some metrics for evaluating a model. The most popular metrics among these metrics is,

Miss Classificatio Error (MCE)

$$Accuracy = 1 - MCE$$

F1 Score

Validation Set Approach

- Split the dataset into two sets,
 - Training dataset (Generally 80% of the data But it can be changed)
 - Testing dataset (Rest of the data)
- Train the model using the training dataset and then check the Accuracy & the Miss Classification Error of the model using the testing dataset. Above mentioned metrics of the model will be calculated, and the model will be evaluated.

Confusion Matrix

- For evaluating the above discussed metrics, a special tool is used which is called as Confusion Matrix.
- Consider the following example where we have to predict a categorical variable with two levels; Yes & No.
- Consider that we have predicted 100 testing observations using a trained model. Then the confusion matrix can be obtained as,

Predicted Output	Actual Output		
		Yes	No
	Yes	40	10
	NO	20	30

Confusion Matrix

	Actual Output		
Predicted Output		Yes	No
	Yes	40	10
	NO	20	30

$$Accuracy = \frac{40 + 30}{100} \times 100\% = 70\%$$

Confusion Matrix

	Actual Output		
Predicted Output		Yes	No
	Yes	40	10
	NO	20	30

$$MCE = \frac{10 + 20}{100} \times 100\% = 30\%$$

$$Accuracy = 1 - MCE$$

Confusion Matrix

Predicted Output	Actual Output		
		Yes	No
	Yes	True Positives (TP)	False Positives (FP)
	No	False Negatives (FN)	True Negatives (TN)

$$\text{Precision} = \frac{\text{True Positive}}{\text{Actual Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

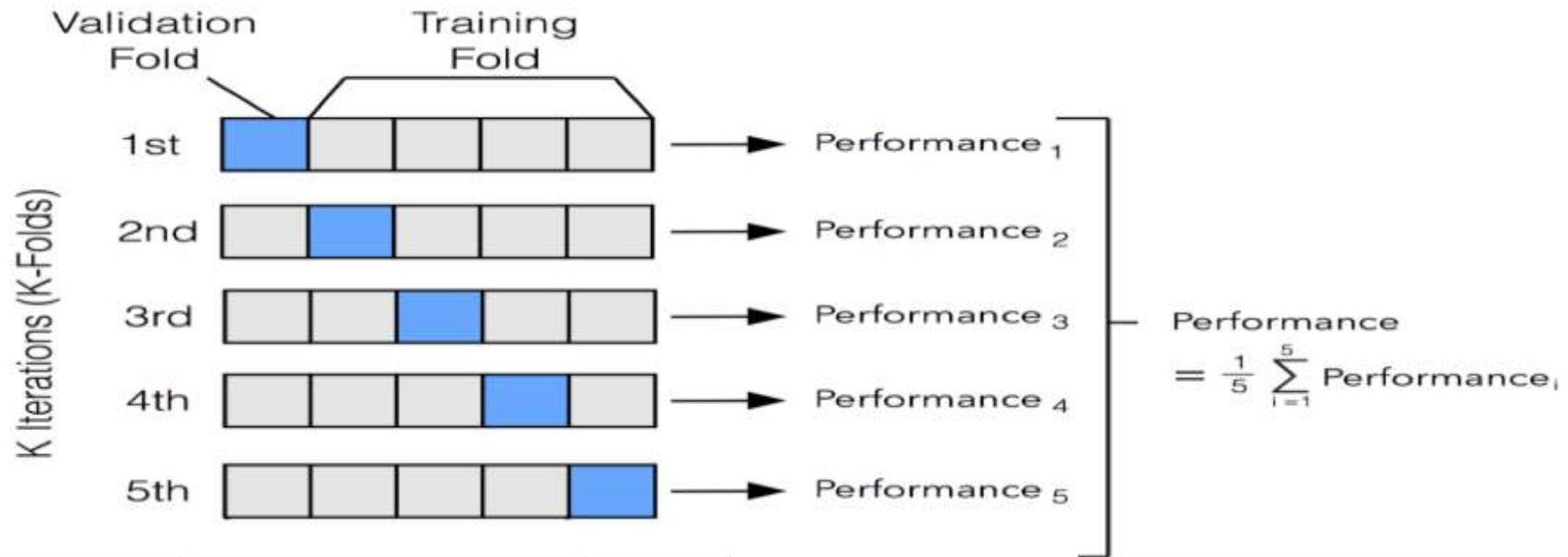
$$\text{Recall} = \frac{\text{True Positive}}{\text{Predicted Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

$$\text{Accuracy} = \frac{\text{True Positive} + \text{True Negative}}{\text{Total}}$$

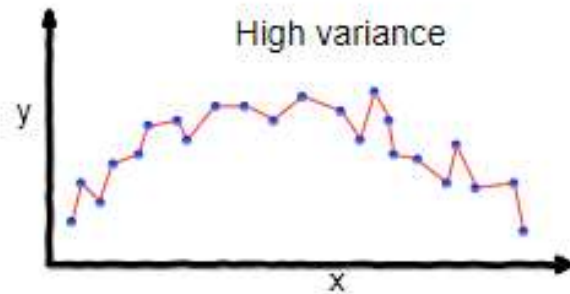
$$F_1 = \left(\frac{\text{recall}^{-1} + \text{precision}^{-1}}{2} \right)^{-1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Cross Validation

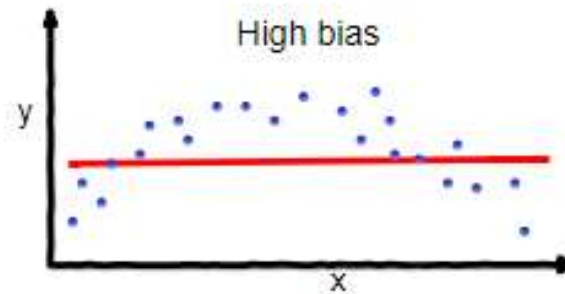
- Cross-validation is a technique in machine learning where data is divided into subsets.
- Models are trained on some and tested on others, ensuring robust performance evaluation.



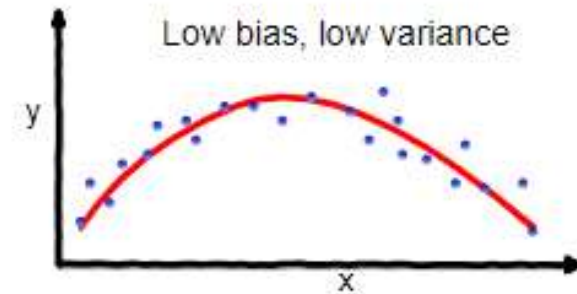
Overfitting & Underfitting



overfitting



underfitting



Good balance

Underfitting

- Underfitting refers to a model that can neither model the training data nor generalize to new data.
- An underfit machine learning model is not a suitable model and will be obvious as it will have poor performance on the training data.

Overfitting

- Overfitting refers to a model that models the training data too well.
- This means that the noise or random fluctuations in the training data is picked up and learned as concepts by the model.
- The problem is that these concepts do not apply to new data and negatively impact the model's ability to generalize.

Overfitting

- **Complex Model:**
 - Models with a large number of parameters can memorize the training data, including its noise, leading to overfitting.
 - Use simpler models with fewer parameters, or apply regularization techniques to penalize complex models.
- **Insufficient Training Data:**
 - Models, especially deep learning models, require a large amount of data to learn meaningful patterns. Insufficient data can lead to overfitting.
 - Gather more labeled data if possible. If not, consider data augmentation techniques to artificially increase the size of the training dataset.

Overfitting

- Leakage from Test Set:
 - If information from the test set is used during the training process (e.g., in feature selection or hyperparameter tuning), the model can overfit the test set.
 - Strictly separate the training, validation, and test datasets. Ensure that the model is not exposed to the test set until the final evaluation.
- Too Many Epochs in Training:
 - Training a model for too many epochs can lead to overfitting because the model starts memorizing the training data after it has already learned the underlying patterns.
 - Use early stopping techniques to halt the training process when the model's performance on a validation set starts degrading.

Solutions for Overfitting

- Regularization
- Ensemble Learning
- Feature Selection
- Feature Extraction

Class Imbalance

- Class imbalance is a situation that occurs when the number of instances in each class of a dataset is not equal.
- In other words, some classes have significantly more instances than others.
- For example, in a binary classification problem where one class is the positive class and the other is the negative class, if the positive class has only a few instances compared to the negative class, then the problem is said to be imbalanced.

Handling Class Imbalance

- There are numerous techniques to handle class imbalance.
 - Undersampling
 - Oversampling
 - Ensemble Method

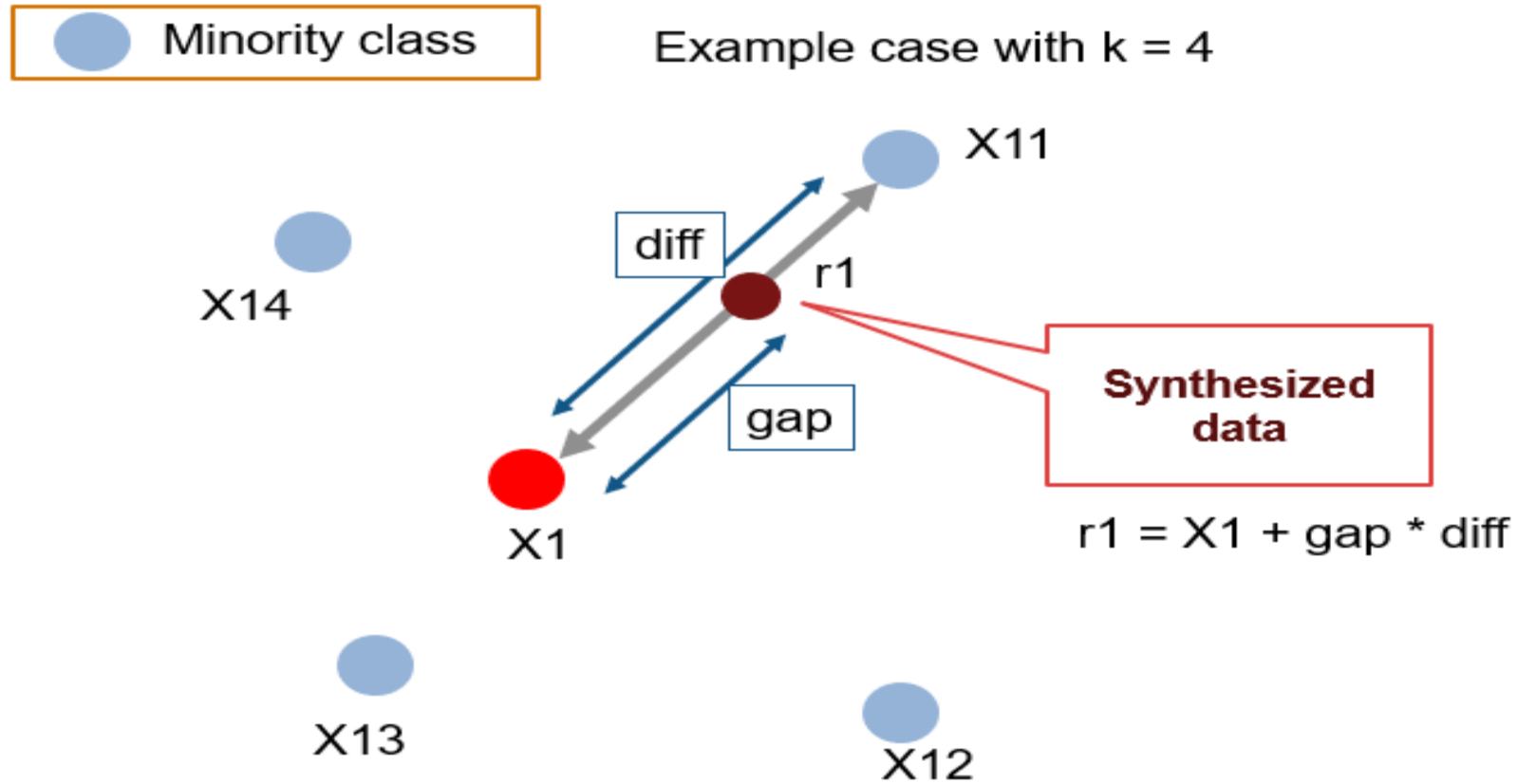
Undersampling

- We reduce the number of samples from the majority class to balance the class distribution in the training data.
- It is a simple technique and since we reduce the size of the data that we push into the models, we reduce training time and memory requirements.

Oversampling

- In the opposite way of undersampling, now we create more duplicates of the minority class.
- There are various ways to oversample, but the most famous one is called Synthetic Minority Over-sampling Technique (SMOTE).
- SMOTE generates synthetic samples for the minority class by interpolating new points between existing ones.
- For each sample in the minority class, it selects k nearest neighbors from the same class.

Oversampling



Oversampling

- When we use SMOTE we do not lose information since this method does not remove any sample.
- It reduces the risk of overfitting because it generates synthetic samples that are not identical to the original samples.
- On the other hand, SMOTE generates synthetic samples, which can introduce noise and outliers in the data. If the dataset is large, then applying SMOTE can be costly.

Ensemble Methods

- Ensemble techniques can be used to improve the classification accuracy of minority classes.
- Balanced Random Forest (BRF) classifier is an extension of the traditional Random Forest algorithm designed to handle imbalanced datasets.
 - Bootstrap Sampling
 - Balancing Class Weights - In Balanced Random Forest, during the training process of each decision tree, the algorithm assigns higher weights to minority class samples. This means that errors made on minority class instances are penalized more, encouraging the algorithm to pay more attention to these instances.
 - Voting/Averaging